



۸۱۰۱۰۱۵۵۷

۸۱۰۱۰۱۴۸۰



تمرین ۲

سیستم های توزیع شده

نام و نام خانوادگی: نرگس بابالار، برنا فروهری

بخش اول: آشنایی با RPC و فناوری های مختلف آن

۱) تعریف RPC (Remote Procedure Call)

RPC یک الگوی ارتباطی در سیستم های توزیع شده است که اجازه می دهد برنامه ی روی یک ماشین (Client) تابع یا رویه ای را که روی ماشین دیگری (Server) قرار دارد، طوری صدا بزند که انگار یک تابع محلی است. در ظاهر، برنامه نویس فقط نام تابع را می بیند و پارامترها را به آن پاس می دهد و نتیجه را تحویل می گیرد؛ اما در پشت صحنه، یک سری مکانیزم شبکه ای و سیستم عاملی درگیر می شوند تا این فراخوانی از طریق شبکه انجام شود. در مدل RPC، توسعه دهنده به جای اینکه خودش مستقیماً با سوکت ها، پروتکل های شبکه و فرمت بندی پیام ها (Message Formatting) سروکار داشته باشد، یک «لایه ی انتزاعی» در اختیار دارد. این لایه جزئیات مثل باز کردن اتصال، ارسال داده ها، منتظر ماندن برای پاسخ، مدیریت خطاهای شبکه و... را پنهان می کند. به همین دلیل، RPC برای ساخت سرویس ها، میکروسرویس ها و نرم افزارهای توزیع شده ی پیچیده بسیار محبوب است. یکی از اهداف اصلی RPC، فراهم کردن شفافیت مکانی (Location Transparency) است؛ یعنی فراخواننده (Client) تا حد زیادی از این که تابع روی همان ماشین اجرا می شود یا روی ماشینی دوردست، بی خبر می ماند. این شباهت با فراخوانی محلی در عین حال هم مزیت است (سادگی توسعه) و هم چالش (امکان نادیده گرفتن محدودیت ها و ریسک های شبکه). در معماری های مدرن، RPC معمولاً روی پروتکل هایی مثل TCP یا HTTP پیاده سازی می شود. فناوری هایی نظیر gRPC، JSON-RPC و XML-RPC نمونه هایی از پیاده سازی های عملی این

ایده هستند که هر کدام با فرمت پیام، کارایی و امکانات خاص خود، همین مفهوم «فراخوانی تابع از راه دور» را ارائه می کنند.

۲ ایده‌ی اصلی فراخوانی تابع از راه دور

ایده‌ی اصلی RPC این است که شما بتوانید تابعی را که روی یک فرآیند دیگر (و احتمالاً روی یک ماشین دیگر) اجرا می شود، مثل یک تابع عادی زبان برنامه نویسی تان صدا بزنید؛ بدون این که مجبور شوید خودتان جزئیات لایه‌ی انتقال (Transport) و پروتکل را مدیریت کنید. برای رسیدن به این هدف، معمولاً مکانیزم زیر به کار می رود:

□ تعریف رابط (Interface) یا سرویس: ابتدا روی سرور مجموعه‌ای از توابع (یا متدها) به عنوان «رابط سرویس» تعریف می شود. در سیستم‌های پیشرفته تر، این رابط در قالب یک IDL (Interface Definition Language) مستقل از زبان بیان می شود.

□ تولید Stub های سمت Client و Server: بر اساس این رابط، ابزار RPC به صورت خودکار دو قطعه کد تولید می کند:

□ Client Stub: از دید برنامه نویس، این کد شبیه یک تابع عادی است؛ اما در واقع مسئول بسته بندی پارامترها (Marshalling)، ارسال آن ها از طریق شبکه و دریافت پاسخ است.

□ Server Stub: روی سرور قرار دارد و پس از دریافت پیام شبکه، داده ها را باز می کند (Unmarshalling)، تابع واقعی را صدا می زند و نتیجه را دوباره در قالب پیام روی شبکه برمی گرداند.

□ Marshalling و Unmarshalling: تبدیل پارامترها به یک قالب قابل ارسال روی شبکه را Marshalling می گویند. در سمت سرور، عکس این عمل انجام می شود که به آن Unmarshalling می گویند.

□ ارسال پیام روی شبکه و مدیریت پاسخ: Client Stub داده‌ی مارشال شده را از طریق یک پروتکل (مثلاً TCP یا HTTP/2) به آدرس سرور می‌فرستد.

□ مدیریت خطا و زمان‌بندی (Timeout): چون این فراخوانی از طریق شبکه انجام می‌شود، باید سناریوهایی مثل قطع ارتباط، تأخیر زیاد، پاسخ ندادن سرور و غیره را در نظر گرفت.

۳ تفاوت فراخوانی محلی و فراخوانی راه دور

اگرچه هدف RPC این است که فراخوانی راه دور را شبیه فراخوانی محلی نشان دهد، اما در واقع این دو از نظر رفتار، هزینه و ریسک تفاوت‌های مهمی دارند. فهم این تفاوت‌ها برای طراحی صحیح سیستم‌های توزیع شده ضروری است.

۱.۳ هزینه‌ی زمانی (Latency) و کارایی

فراخوانی محلی (Local Call)

در فراخوانی محلی، اجرای تابع در همان فرآیند (یا نهایتاً همان ماشین) انجام می‌شود. هزینه‌ی این فراخوانی معمولاً فقط شامل کار CPU، دسترسی به حافظه و یک مقدار ناچیز مدیریت پشته است. زمان اجرا در حد نانو ثانیه تا میکروثانیه است.

فراخوانی راه دور (Remote Call)

در RPC، هر فراخوانی نیازمند:

- مارشال کردن پارامترها،
- ارسال بسته‌ها از طریق شبکه،
- احتمال عبور از چند روتر/سوییچ،
- دریافت و آنمارشال داده در سمت سرور،

□ انجام محاسبه،

□ مارشال و ارسال پاسخ و

□ آنمارشال پاسخ در سمت کلاینت است.

هر کدام از این مراحل تأخیر اضافه می کنند. در نتیجه زمان پاسخ از میلی ثانیه تا حتی چند ثانیه می تواند متغیر باشد. بنابراین تعداد و فرکانس فراخوانی ها در طراحی RPC بسیار مهم است و نمی توان مثل فراخوانی محلی، بدون فکر آن ها را در حلقه های پرتکرار قرار داد.

۲.۳ قابلیت اعتماد (Reliability)

محلی:

در محیط محلی، اگر فرآیند در حال اجرا باشد و خطای خاصی رخ ندهد، تقریباً همیشه می توان روی موفق بودن فراخوانی حساب کرد. شکست ها معمولاً به خطای منطقی برنامه یا Crash محدود می شوند.

راه دور:

در فراخوانی شبکه ای، عوامل زیادی ممکن است باعث شکست شوند: قطع شدن شبکه، در دسترس نبودن سرور، Timeout، ازدحام شبکه، مشکلات DNS و بنابراین فراخوانی راه دور همیشه باید با دید «شاید شکست بخورد» طراحی شود. در نتیجه، مدیریت خطا، Retry، Circuit Breaker و لاگ کردن رفتار سیستم اهمیت زیادی پیدا می کند.

۳.۳ معنای خطا و استثناها

محلی:

خطای تابع معمولاً دقیقاً در همان نقطه اجرا رخ می دهد و برنامه نویس با استثناها (Exception) یا کدهای خطا سروکار دارد که نسبتاً قابل پیش بینی هستند.

راه دور:

علاوه بر خطاهای منطقی تابع سرویس دهنده، خطاهای شبکه و زیرساخت نیز باید گزارش شوند. بنابراین در RPC، لایه‌ی واسط (Stub/Runtime) معمولاً نوع خاصی از خطاها را تعریف می‌کند تا تمایز بین «خطای شبکه» و «خطای منطق سرویس» مشخص باشد. این موضوع طراحی قرارداد خطا (Error Contract) را پیچیده‌تر می‌کند.

۴.۳ شفافیت مکانی و ریسک آن

فراخوانی محلی ذاتاً در همان مکان انجام می‌شود و برنامه‌نویس از هزینه‌ی آن آگاه است. در RPC، چون کد سمت کلاینت ظاهر «محلی» دارد، ممکن است توسعه‌دهنده:

□ تعداد بسیار زیادی فراخوانی ریزدانه (Chatty) به سرویس انجام دهد،

□ یا فراخوانی را در حلقه‌ای قرار دهد که بار شبکه را چند برابر می‌کند.

این موضوع ممکن است باعث افت شدید کارایی شود و به همین دلیل گفته می‌شود شبیه بودن RPC به فراخوانی محلی «هم مفید و هم خطرناک» است.

۵.۳ مرزهای شکست (Failure Boundaries) و تراکنش‌ها

در فراخوانی محلی، معمولاً یا کل تابع موفق می‌شود یا برنامه Crash می‌کند؛ اما مفهوم «نیمه‌انجام‌شدن» ناشی از مشکلات ارتباطی وجود ندارد. در فراخوانی راه دور، ممکن است تابع در سمت سرور اجرا شده باشد ولی پاسخ به کلاینت نرسد؛ یا برعکس، درخواست هرگز به سرور نرسیده باشد. تشخیص این سناریوها همیشه آسان نیست و طراحی عملیات‌های Idempotent (قابل تکرار بدون اثر منفی) در RPC اهمیت زیادی دارد.

۶.۳ استقلال و نسخه‌بندی (Versioning)

در کد محلی، همه چیز در یک باینری یا یک مجموعه مازول مشترک کامپایل می‌شود و معمولاً نسخه‌ی کدها هماهنگ است. در RPC، کلاینت و سرور می‌توانند روی نسخه‌های مختلفی از

سرویس کار کنند. این موضوع ما را مجبور می کند به سازگاری نسخه ها (Backward/Forward Compatibility) و طراحی قراردادهای پایدار بیشتر فکر کنیم.

۴ نقش Server Stub و Client Stub

در معماری RPC، Stub ها نقش واسطه های کلیدی را بین کد اصلی برنامه (Application Code) و زیرساخت ارتباطی شبکه ایفا می کنند. این Stub ها به گونه ای طراحی شده اند که فراخوانی تابع از راه دور را برای برنامه نویس شفاف کنند و پیچیدگی های ارتباط شبکه را پنهان نمایند.

۱.۴ Client Stub (یا Proxy)

وظیفه اصلی: Client Stub شبیه یک «پروکسی» برای تابع (یا متد) اصلی است که روی سرور قرار دارد. این Stub در سمت کلاینت اجرا می شود و دارای همانی (Signature) تابع اصلی است. نحوه عملکرد:

□ دریافت فراخوانی: وقتی برنامه کلاینت، تابعی را صدا می زند، در واقع Client Stub را صدا زده است.

□ بسته بندی پارامترها (Marshalling): Stub پارامترهای دریافتی را جمع آوری کرده و آن ها را به فرمتی تبدیل می کند که قابل ارسال از طریق شبکه باشد. این فرآیند معمولاً شامل تبدیل انواع داده ی مختلف (مثل اعداد، رشته ها، آرایه ها، ساختارها) به یک دنباله ی خطی از بایت ها (Byte Stream) است.

□ ارسال پیام: Stub سپس این دنباله ی بایت را به همراه اطلاعات لازم (مانند نام تابع، شناسه درخواست و...) از طریق پروتکل ارتباطی (مثلاً TCP یا HTTP) به سرور ارسال می کند.

□ انتظار و دریافت پاسخ: پس از ارسال، Stub منتظر دریافت پاسخ از سرور می ماند.

□ باز کردن پاسخ (Unmarshalling): وقتی پاسخ (شامل نتیجه‌ی اجرای تابع یا اطلاعات خطا) دریافت شد، Stub آن را دریافت کرده و دنباله‌ی بایت را دوباره به فرمت قابل استفاده برای برنامه کلاینت (مانند مقدار بازگشتی تابع) تبدیل می‌کند.

□ برگرداندن نتیجه: در نهایت، Stub نتیجه‌ی بازگشتی یا خطای رخ داده را به برنامه کلاینت برمی‌گرداند.

۲.۴ Server Stub (یا Skeleton)

وظیفه اصلی: Server Stub (که گاهی Skeleton نامیده می‌شود) در سمت سرور قرار دارد و مسئول دریافت درخواست‌های ارسالی از کلاینت‌ها، فراخوانی تابع واقعی سرویس و ارسال نتایج به کلاینت است.
نحوه عملکرد:

□ دریافت پیام: Server Stub به طور مداوم به شبکه گوش می‌دهد و پیام‌های دریافتی از کلاینت‌ها را دریافت می‌کند.

□ باز کردن پیام (Unmarshalling): Stub پیام دریافتی را تجزیه کرده و دنباله‌ی بایت را به پارامترهای تابع اصلی تبدیل می‌کند.

□ فراخوانی تابع اصلی: با استفاده از پارامترهای تبدیل شده، Stub تابع سرویس واقعی را که روی سرور پیاده‌سازی شده است، صدا می‌زند.

□ بسته‌بندی نتیجه (Marshalling): نتیجه‌ی بازگشتی از تابع اصلی (یا اطلاعات خطا) توسط Stub دریافت و به دنباله‌ی بایت قابل ارسال روی شبکه تبدیل می‌شود.

□ ارسال پاسخ: Stub این دنباله‌ی بایت را از طریق شبکه به سمت کلاینت مبدأ ارسال می‌کند.

خلاصه:

Stub ها با ترکیب Client Stub و Server Stub، یک «تونل» شفاف برای ارتباط بین کلاینت و سرور ایجاد می کنند. آن ها ارتباط شبکه، تبدیل داده ها و مدیریت پروتکل را از دید برنامه نویس پنهان نگه می دارند و امکان پیاده سازی مدل «فراخوانی تابع» را به صورت توزیع شده فراهم می سازند. این بخش از معماری RPC، مهم ترین بخش و سهولت استفاده از این الگوی ارتباطی است.

۵ مفهوم Serialization و Deserialization

در سیستم های توزیع شده، ارتباط بین اجزای مختلف (مانند کلاینت و سرور) معمولاً از طریق شبکه ی کامپیوتری انجام می شود. شبکه ها داده ها را به صورت بسته هایی از بایت ها (Byte Streams) منتقل می کنند. در حالی که برنامه های ما با ساختارهای داده ای پیچیده ای مانند اشیاء (Objects)، ساختارها (Structs)، لیست ها (Lists) و... کار می کنند. بنابراین، برای ارسال این ساختارهای داده ای پیچیده از یک فرآیند به فرآیند دیگر (خصوصاً اگر روی ماشین های مختلف باشند)، نیاز به یک مکانیزم تبدیل داریم. اینجاست که مفاهیم Serialization و Deserialization وارد عمل می شوند.

۱.۵ Serialization (سریال سازی / بایت سازی)

تعریف: سریال سازی به فرآیند تبدیل یک ساختار داده ای یا وضعیت یک شیء (Object) در حافظه ی یک برنامه به یک فرمت قابل ذخیره سازی یا قابل انتقال (معمولاً دنباله ی خطی از بایت ها) گفته می شود.

هدف: این فرمت استاندارد شده (که می تواند JSON، XML، Protocol Buffers، یا فرمت های باینری اختصاصی باشد) می تواند به راحتی از طریق شبکه ارسال شود، در یک فایل ذخیره گردد، یا در پایگاه داده نگهداری شود.

نحوه ی عملکرد: در زمان سریال سازی، مقادیر فیلدها یا خصوصیات یک شیء، همراه با ساختار کلی

آن (و گاهی اطلاعات نوع داده) جمع‌آوری شده و به ترتیب مشخصی به دنباله‌ای از بایت‌ها تبدیل می‌شوند.

مثال: فرض کنید کلاسی به نام User با فیلدهای name (رشته) و age (عدد صحیح) داریم. در زمان سریال‌سازی، مقادیر name و age همراه با ساختار کلی User به یک رشته‌ی متنی (مثلاً "{name: 'Alice', age: 30}") یا دنباله‌ای از بایت‌های باینری تبدیل می‌شوند.

۲.۵ Deserialization (دی‌سریال‌سازی / بایت‌گشایی)

تعریف: دی‌سریال‌سازی عکس فرآیند سریال‌سازی است. این فرآیند، دنباله‌ی بایت‌ها یا فرمت قابل انتقال را دریافت کرده و آن را دوباره به ساختار داده‌ای اصلی یا وضعیت قابل استفاده در حافظه‌ی برنامه تبدیل می‌کند.

هدف: پس از دریافت داده‌ها از شبکه یا خواندن از فایل، باید بتوانیم آن‌ها را دوباره به صورت اشیاء یا ساختارهای داده‌ای قابل فهم برای برنامه درآوریم تا بتوانیم با آن‌ها کار کنیم. نحوه‌ی عملکرد: در زمان دی‌سریال‌سازی، فرمت دریافتی تجزیه شده و بر اساس ساختار تعریف شده (که باید با ساختار زمان سریال‌سازی سازگار باشد)، اشیاء و مقادیر داده‌ای در حافظه بازسازی می‌شوند.

مثال: اگر رشته‌ی "{name: 'Alice', age: 30}" را دریافت کنیم، فرآیند دی‌سریال‌سازی آن را به یک شیء User با مقادیر name="Alice" و age=30 تبدیل می‌کند.

۳.۵ اهمیت در RPC

در RPC، سریال‌سازی و دی‌سریال‌سازی جزء مهمی از فرآیند هستند:

□ در سمت کلاینت: Client Stub پارامترهای تابع را سریال‌سازی می‌کند تا بتواند آن‌ها را به سرور بفرستد.

□ در سمت سرور: Server Stub پیام دریافتی را دی‌سریال‌سازی می‌کند تا بتواند پارامترها را

استخراج کرده و تابع واقعی را فراخوانی کند. سپس نتیجه‌ی تابع را سریال‌سازی می‌کند تا بتواند آن را به کلاینت برگرداند.

□ در سمت کلاینت (پاسخ): Client Stub پاسخ دریافتی را دی‌سریال‌سازی می‌کند تا نتیجه را به برنامه کلاینت برگرداند.

این فرآیند تبدیل داده‌ها، معمولاً توسط کتابخانه‌های استاندارد زبان برنامه‌نویسی یا توسط ابزارهای مخصوص پروتکل RPC (مانند Protocol Buffers برای gRPC) انجام می‌شود. انتخاب فرمت سریال‌سازی (مثل JSON، XML، Protocol Buffers، MessagePack) تأثیر مستقیمی بر کارایی (سرعت و حجم داده)، خوانایی و قابلیت توسعه‌پذیری سیستم توزیع‌شده دارد.

۶ مفهوم IDL یا Interface Definition Language

در سیستم‌های مبتنی بر RPC، کلاینت و سرور معمولاً می‌توانند با زبان‌ها و پلتفرم‌های متفاوتی پیاده‌سازی شوند. برای این‌که این اجزا بتوانند بدون وابستگی مستقیم به زبان برنامه‌نویسی یکدیگر، با هم صحبت کنند، نیاز به یک «قرارداد مشترک» داریم. IDL (Interface Definition Language) زبان یا قالبی است که این قرارداد را به صورت دقیق و مستقل از زبان (Language-Neutral) توصیف می‌کند.

۱.۶ IDL چیست؟

IDL زبانی است برای تعریف:

□ سرویس‌ها (Services)

□ توابع/متدها (Methods)

□ نوع پارامترها و مقادیر بازگشتی (Types)

□ ساختارهای داده‌ای (Enums, Messages, Structs)

این تعریف معمولاً در یک فایل متنی مستقل (مثل proto در gRPC) نوشته می‌شود و به عنوان (Single Source of Truth) برای کلاینت و سرور عمل می‌کند.

۲.۶ نقش IDL در RPC

تعریف قرارداد مستقل از زبان: با IDL، ابتدا فقط قرارداد سرویس را تعریف می‌کنیم: چه متدی وجود دارد، نامش چیست، چه ورودی و خروجی‌ای دارد. این تعریف وابسته به زبان خاصی مثل Go، Java یا Python نیست.

تولید خودکار Stubها: ابزارهای مرتبط با هر فناوری RPC، از روی فایل IDL، کدهای مورد نیاز را تولید می‌کنند:

□ Client Stub (Proxy)

□ Server Stub (Skeleton)

□ انواع داده‌ای (Classes / Structs) متناظر با پیام‌ها

به این ترتیب، توسعه‌دهنده به جای نوشتن دستی کد سریال‌سازی، Stub و... فقط روی منطق کسب‌وکار تمرکز می‌کند.

هماهنگی نسخه‌ها (Versioning):

وقتی قرارداد سرویس در یک مکان متمرکز (فایل IDL) تعریف شده باشد، اعمال تغییرات کنترل شده در API و نگهداری سازگاری (Backward Compatibility) آسان‌تر می‌شود. می‌توان فیلدهای جدید اضافه کرد، فیلدهای قدیمی را Deprecated کرد و....

تسهیل ارتباط بین‌زبانی (Interoperability):

از یک فایل IDL می‌توان برای تولید کد در زبان‌های مختلف استفاده کرد (مثلاً از یک proto. هم Stub برای Go گرفت، هم برای Java، هم برای Python). در نتیجه یک سرویس نوشته شده با Go می‌تواند به راحتی توسط یک کلاینت Node.js مصرف شود.

۷ خطاهای رایج در RPC

فراخوانی راه دور ذاتاً شکننده تر از فراخوانی محلی است؛ چون پای شبکه، یک ماشین، و چند جزء مستقل در میان است. خطاهای رایج در RPC را می توان در چند دسته ی اصلی خلاصه کرد:

۱.۷ خطاهای ارتباطی (Communication Errors)

Timeout (پایان مهلت): کلاینت درخواست را می فرستد اما در زمان تعیین شده پاسخی نمی رسد. ممکن است: سرور مشغول باشد یا کند عمل کند، شبکه دچار تأخیر یا قطع موقت شده باشد، یا درخواست اصلاً به سرور نرسیده باشد. طراحی مناسب Timeout و سیاست های Retry برای مدیریت این خطا بسیار مهم است.

Connection Failure (عدم برقراری اتصال): کلاینت اصلاً نمی تواند به سرور متصل شود (IP اشتباه، پورت بسته، Firewall، سرور Down و...). در این حالت، فراخوانی RPC حتی آغاز هم نمی شود.

Network Partition / قطع بخش شبکه: ممکن است برخی ماشین ها به هم دسترسی داشته باشند و برخی دیگر نه. از دید کلاینت مثل قطع کامل سرور به نظر می رسد، در حالی که سرور برای بقیه ی نودها در دسترس است. این نوع شکست ها در سیستم های توزیع شده رایج هستند.

۲.۷ خطاهای معنایی و سازگاری (Compatibility Errors / Semantic)

عدم سازگاری نسخه ها (**Version Mismatch**): اگر کلاینت و سرور روی نسخه های متفاوتی از API یا IDL کار کنند، ممکن است: کلاینت متدی را صدا بزند که دیگر روی سرور وجود ندارد، فرمت پیام تغییر کرده باشد (فیلد حذف شده یا نوع داده تغییر کرده باشد)، یا سرور نتواند پیام کلاینت را به درستی دی سریال سازی کند.

Errors Deserialization / Serialization: عدم تطابق نوع داده ها، مقداردهی نادرست فیلدهای اجباری، یا خراب شدن داده (**Corruption**) در مسیر انتقال می تواند باعث خطای سریال سازی

شود.

۳.۷ خطاهای مرتبط با معنای اجرا (Semantic Errors / Execution)

خطای منطق سرویس (Application Error): تابع سمت سرور ممکن است به دلیل ورودی نامعتبر، خطای تجاری، نبودن داده در پایگاه داده، خطای اعتبارسنجی و... شکست بخورد. این خطاها از دید RPC «موفق با خطای منطقی» محسوب می شوند؛ یعنی ارتباط برقرار شده ولی سرویس پیام خطا برمی گرداند.

Partial Failure / نیمه انجام شدن عملیات: ممکن است سرور عملیات را انجام داده باشد (مثلاً پول را از حسابی کم کرده) اما پاسخ به کلاینت نرسد؛ یا برعکس، درخواست اصلاً به سرور نرسیده باشد ولی کلاینت تصور کند شاید رسیده باشد. این سناریوها تشخیص را دشوار می کنند و طراحی عملیات های Idempotent و استفاده از الگوهایی مثل Transactional Outbox را ضروری می سازند.

Resource Exhaustion (کمبود منابع): اگر سرور با حجم زیاد درخواست ها روبه رو شود، ممکن است منابعی مثل Thread، حافظه یا Connection ها تمام شده و خطاهایی مثل Too Many Requests یا OutOfMemory رخ دهد.

۴.۷ خطاهای طراحی / معماری

Chatty RPC (فراخوانی ریزدانه ی زیاد): طراحی بد API که نیازمند ده ها یا صدها فراخوانی RPC برای انجام یک کار منطقی ساده است، باعث افزایش تأخیر، مصرف پهنای باند و احتمال خطا می شود.

وابستگی بیش از حد به شفافیت مکانی: رفتار RPC را بسیار شبیه فراخوانی محلی در نظر گرفتن و استفاده ی بدون ملاحظه از آن در حلقه های پرتکرار، باعث ایجاد گلوگاه (Bottleneck) و شکست های زنجیره ای می شود.

۸ مقایسه‌ی REST با RPC

هر دو رویکرد RPC و REST برای ایجاد سرویس‌های شبکه‌ای و API استفاده می‌شوند، اما فلسفه و شیوه‌ی طراحی آن‌ها متفاوت است.

۱.۸ مدل مفهومی

RPC (Remote Procedure Call): محوریت بر «فراخوانی تابع/متد» دارد. API معمولاً به صورت «سرویس + متد» طراحی می‌شود؛ مانند `AuthService.Login`، `UserService.CreateUser`. تمرکز روی عملیات‌ها (Actions) است.

REST (Representational State Transfer): محوریت بر «منابع (Resources)» دارد. از URI‌ها برای اشاره به منابع استفاده می‌کند، مثل `/users/123`، `/orders/42/items`. از متدهای استاندارد HTTP (`GET`، `POST`، `PUT`، `DELETE`، `PATCH`، ...) برای اعمال روی این منابع بهره می‌گیرد. تمرکز روی نمایش و انتقال وضعیت منابع است، نه روی فراخوانی تابع مشخص.

۲.۸ پروتکل و فناوری زیرساخت

RPC: می‌تواند روی پروتکل‌های مختلفی پیاده شود: TCP خام، HTTP/1.1، HTTP/2 و برخی پیاده‌سازی‌ها مثل gRPC از HTTP/2 با فرمت باینری استفاده می‌کنند. معمولاً از IDL (مثل proto) برای تعریف سرویس استفاده می‌کند.

REST: معمولاً روی HTTP/1.1 یا HTTP/2 سوار است، اما مبتنی بر قواعد استاندارد HTTP است. به جای IDL رسمی، اغلب از مستندات OpenAPI/Swagger/JSON برای توصیف API استفاده می‌شود (که نوعی قرارداد غیررسمی محسوب می‌گردد).

۳.۸ سبک داده و فرمت پیام

RPC: اغلب از فرمت های باینری کارآمد مانند Thrift, Protocol Buffers یا فرمت های اختصاصی استفاده می کند. این فرمت ها حجم کمتر و سرعت بیشتر در سریال سازی/دی سریال سازی دارند، اما برای انسان ها کمتر خوانا هستند. در نسخه های ساده تر (مثل JSON-RPC)، ممکن است از JSON استفاده شود.

REST: بیشتر با JSON و در گذشته با XML همراه است. پیام ها برای انسان ها نسبتاً خوانا هستند و Debug کردن آن ها ساده تر است. با ابزارهای عمومی HTTP (curl، مرورگر، Postman) به راحتی قابل آزمایش و مشاهده اند.

۴.۸ استفاده ها (Use Cases)

RPC مناسب است برای: ارتباط داخلی بین سرویس های یک مجموعه (Microservices) که نیازمند کارایی بالا و قرارداد دقیق هستند؛ سناریوهایی که تعداد زیادی فراخوانی با Payload کوچک نیاز است؛ محیط هایی با کنترل کامل روی 2 طرف ارتباط (کلاینت و سرور).
REST مناسب است برای: API های عمومی (Public APIs) که کاربران و زبان های متنوعی دارند؛ جایی که سادگی، استاندارد بودن و خوانایی مهم تر از کارایی حداکثری است؛ سیستم هایی که ارتباط آن ها بیشتر حول «منابع» و CRUD می چرخد (ایجاد، خواندن، به روزرسانی، حذف).

۵.۸ مزایا و معایب نسبی

RPC: مزایا: کارایی بالا (به خصوص با فرمت های باینری مثل Protobuf)؛ (Strongly Typed) از طریق IDL؛ امکان پشتیبانی از Streaming دوطرفه (مانند gRPC). معایب: Debug و مشاهده پیام ها سخت تر است (به خاطر فرمت باینری و پروتکل های پیشرفته تر)؛ برای کلاینت هایی که خارج از اکوسیستم طراحی شده هستند، ممکن است کار با آن دشوارتر باشد (نیاز به تولید Stub و ابزارهای خاص).

REST: مزایا: سادگی و استفاده‌ی گسترده؛ تشخیص و Debug راحت به خاطر استفاده از HTTP و JSON؛ سازگاری عالی با وب و مرورگرها، و امکان مصرف آسان توسط انواع کلاینت‌ها. معایب: قراردادی ضعیف‌تر نسبت به RPC (اگرچه OpenAPI کمک می‌کند، ولی معمولاً به استحکام IDL‌های سنتی نمی‌رسد)؛ کارایی پایین‌تر نسبت به فرمت‌های باینری در سناریوهای حساس به تأخیر یا حجم داده؛ پشتیبانی پیچیده‌تر برای Streaming دوطرفه.

۹ مقایسه فناوری‌های RPC

در این بخش، سه فناوری مطرح در حوزه‌ی فراخوانی از راه دور شامل gRPC، JSON-RPC و XML-RPC را بر اساس مجموعه‌ای از معیارهای دقیق و کاربردی بررسی و تحلیل می‌کنیم. معیارهای مقایسه به گونه‌ای انتخاب شده‌اند که علاوه بر جنبه‌های فنی، کارایی عملیاتی، پیچیدگی پیاده‌سازی و میزان تناسب با سیستم‌های توزیع‌شده نیز در آن‌ها پوشش داده شود.

۱.۹ محورهای مقایسه

در ادامه، محورهای مقایسه به صورت فیلدهای مستقل ارائه شده‌اند :

□ نوع داده و فرمت پیام (Data Format): فرمت پیام تعیین می‌کند داده‌ها چگونه در شبکه ارسال شوند. برخی فناوری‌ها مانند gRPC از فرمت‌های باینری مانند Protobuf استفاده می‌کنند که حجم کم و سرعت پردازش بالا دارند. برخی دیگر مانند JSON-RPC و XML-RPC از فرمت‌های متنی (XML/JSON) بهره می‌برند که خوانایی بیشتری دارند اما سربار پردازش و حجم بیشتری ایجاد می‌کنند. انتخاب فرمت پیام تأثیر مستقیم بر کارایی، پهنای باند مصرفی و سرعت تبادل داده‌ها دارد.

□ روش تعریف Interface (نوع قرارداد یا IDL): وجود یا عدم وجود IDL نقش مهمی در استانداردسازی قرارداد میان کلاینت و سرور دارد. فناوری‌هایی مانند gRPC با استفاده از proto. قرارداد دقیقی تعریف می‌کنند و امکان تولید خودکار کد را فراهم می‌سازند. این امر

خطاهای توسعه را کاهش می دهد و باعث سازگاری میان زبان های مختلف می شود. در مقابل، فناوری هایی مانند JSON-RPC الزامی برای قرارداد رسمی ندارند و این موضوع می تواند توسعه را ساده تر اما نگهداری و نسخه پذیری را دشوارتر کند.

□ پشتیبانی از زبان های برنامه نویسی (Language Support): برخی فناوری ها مانند gRPC و Thrift با تولید خودکار کد، پشتیبانی گسترده و قدرتمندی برای زبان های مختلف فراهم می کنند. در مقابل، فناوری هایی مثل RMI تقریباً محدود به زبان Java هستند. میزان تطبیق با زبان های برنامه نویسی مختلف، یک عامل مهم برای تیم هایی است که چند زبان مختلف را هم زمان استفاده می کنند.

□ کارایی و سربار ارتباطی (Performance & Overhead): کارایی شامل سرعت فراخوانی، حجم پیام ها، تأخیر رفت و برگشت و مصرف منابع پردازشی است. فناوری های مبتنی بر فرمت باینری مانند gRPC و Thrift کارایی بسیار بالایی دارند. در مقابل، پروتکل های متنی مانند XML-RPC بیشترین سربار را ایجاد می کنند. این معیار برای سیستم های توزیع شده با بار بالا حیاتی است.

□ سادگی پیاده سازی (Implementation Simplicity): پیچیدگی توسعه، نیازمندی های محیطی، نیاز به ابزارهای جانبی، و حجم تنظیمات در این معیار بررسی می شود. برای مثال، JSON-RPC بسیار ساده و سبک است و بدون نیاز به ابزار اضافی قابل استفاده است. در مقابل، gRPC نیازمند تعریف IDL، تولید کد و تنظیمات بیشتر است، اما ساختارمندتر و مقیاس پذیرتر است.

□ پشتیبانی از جریان داده (Streaming): برخی کاربردها نیازمند ارسال جریان مداوم داده با الگوهای یک طرفه، دوطرفه یا Bidirectional هستند. gRPC مبتنی بر HTTP/2، پشتیبانی کامل از Streaming دارد. فناوری های متنی مانند JSON-RPC و XML-RPC این قابلیت را ندارند و برای داده های بلادرنگ مناسب نیستند.

□ امنیت و قابلیت اطمینان (Security & Reliability): برخی فناوری ها به طور پیش فرض از رمزنگاری، احراز هویت و کنترل نسخه پشتیبانی می کنند. SOAP برای مثال مجموعه ای غنی از استانداردهای امنیتی دارد. gRPC نیز با TLS یکپارچه است. فناوری های ساده تر مانند JSON-RPC فاقد این قابلیت های پیش فرض هستند و امنیت باید به صورت لایه ای اضافه شود.

□ قابلیت نسخه پذیری و تکامل پذیری (Versioning & Evolvability): فناوری هایی مانند gRPC به لطف وجود IDL امکان مدیریت نسخه را بسیار ساده تر می کنند. اما فناوری هایی مانند JSON-RPC در صورت تغییر ساختار پیام ها در معرض بروز ناسازگاری بین سرویس ها هستند.

□ تناسب با سیستم های توزیع شده (Distributed Suitability): برخی فناوری ها برای معماری های میکروسرویس، ارتباطات داخلی سریع و پردازش با حجم بالا طراحی شده اند. برای مثال، gRPC و Thrift در این زمینه عملکرد عالی دارند. در مقابل، فناوری هایی مانند RMI یا XML-RPC بیشتر مناسب محیط های محدود یا سیستم های قدیمی هستند.

□ موارد استفاده رایج (Common Use Cases): هر فناوری معمولاً در یک سری سناریوهای مشخص کاربرد بهتری دارد. به عنوان نمونه: gRPC: ارتباطات داخلی سرویس ها، سیستم های بلادرنگ، میکروسرویس ها JSON-RPC: اپلیکیشن های سبک، افزونه های مرورگر، ارتباطات ساده XML-RPC: ادغام با سیستم های قدیمی (Legacy)

۲.۹ بررسی فناوری ها

۱.۲.۹ gRPC

gRPC یکی از فناوری های مدرن و پرکاربرد در حوزه ی RPC است که توسط Google توسعه داده شده و به طور ویژه برای ارتباطات سریع، ساخت یافته و مقیاس پذیر میان سرویس ها طراحی شده است. این فناوری از Protocol Buffers به عنوان قالب اصلی سریال سازی استفاده می کند.

Protocol Buffers یک فرمت باینری، فشرده و بسیار کارآمد است که در مقایسه با قالب های متنی مانند JSON و XML حجم کمتری تولید می کند و پردازش آن نیز سریع تر انجام می شود. همین ویژگی باعث می شود gRPC برای سیستم هایی که تعداد زیادی درخواست و پاسخ میان اجزای مختلف دارند، گزینه ای بسیار مناسب باشد.

یکی از مهم ترین ویژگی های gRPC استفاده از HTTP/2 به عنوان لایه انتقال است. این موضوع مزایای مهمی مانند Multiplexing، کاهش هزینه ی برقراری اتصال، پشتیبانی از Streaming و مدیریت بهتر ارتباطات هم زمان را فراهم می کند. در نتیجه، gRPC نه تنها برای فراخوانی های ساده ی درخواست-پاسخ مناسب است، بلکه در سناریوهایی که نیاز به ارسال پیوسته ی داده، ارتباط دوطرفه یا تعاملات بلادرنگ وجود دارد نیز عملکرد بسیار خوبی ارائه می دهد. برای نمونه، در سامانه های مانیتورینگ، سرویس های پردازش رویداد، چت بلادرنگ یا ارتباط داخلی میان میکروسرویس ها، این مزیت بسیار مهم است.

از منظر تعریف رابط، gRPC مبتنی بر IDL است و از فایل های proto. برای تعریف سرویس ها، پیام ها و متدها استفاده می کند. این رویکرد باعث می شود قرارداد ارتباطی میان کلاینت و سرور کاملاً شفاف، دقیق و مستقل از زبان برنامه نویسی باشد. سپس با استفاده از ابزارهای تولید کد، نسخه های کلاینت و سرور برای زبان های مختلف تولید می شوند. این ویژگی احتمال خطاهای ناشی از تفسیر نادرست API را کاهش می دهد و نگهداری و توسعه ی سامانه را آسان تر می سازد. البته همین ساختارمندی به این معناست که راه اندازی اولیه ی gRPC نسبت به فناوری های سبک تر کمی پیچیده تر است، زیرا توسعه دهنده باید با مفاهیمی مانند تعریف Schema، تولید کد، مدیریت نسخه و تنظیمات HTTP/2 آشنا باشد.

در مجموع، gRPC برای معماری های Microservices، سامانه های توزیع شده ی پرتراфик، زیرساخت های ابری و کاربردهایی که کارایی، قرارداد رسمی، تایپ بودن دقیق داده ها و قابلیت Streaming در آن ها اهمیت دارد، یکی از بهترین گزینه ها محسوب می شود. نقطه ی قوت اصلی آن در سرعت، ساختارمندی و توسعه پذیری است، هرچند برای API های عمومی و ساده که خوانایی پیام ها و سهولت تست اولویت بیشتری دارند، ممکن است گزینه های سبک تر ترجیح داده شوند.

۲.۲.۹ JSON-RPC

JSON-RPC یک پروتکل سبک و ساده برای پیاده سازی RPC است که از JSON برای قالب بندی پیام ها استفاده می کند. سادگی این فناوری یکی از مهم ترین دلایل محبوبیت آن در پروژه های کوچک، متوسط و کاربردهایی است که نیاز به راه اندازی سریع و بدون پیچیدگی زیاد دارند. از آن جا که JSON یک فرمت متنی، خوانا و به طور گسترده پشتیبانی شده است، پیام های JSON-RPC را می توان به راحتی مشاهده، تولید و اشکال زدایی کرد. این ویژگی باعث می شود توسعه دهندگان بدون نیاز به ابزارهای خاص یا فرایندهای پیچیده ی تولید کد، بتوانند به سرعت یک سرویس RPC سبک ایجاد کنند.

در JSON-RPC معمولاً خبری از IDL اجباری یا قرارداد رسمی سخت گیرانه نیست. این موضوع از یک طرف انعطاف پذیری بالایی ایجاد می کند و روند توسعه را سریع تر می سازد، اما از طرف دیگر ممکن است در پروژه های بزرگ تر مشکل ساز شود. وقتی قرارداد رسمی و ابزار تولید کد وجود نداشته باشد، هماهنگی بین کلاینت و سرور بیشتر به مستندسازی دستی و توافق توسعه دهندگان وابسته می شود. در نتیجه، با بزرگ تر شدن سیستم یا افزایش تعداد تیم ها، احتمال ناسازگاری، خطاهای معنایی و دشواری در نسخه پذیری بیشتر می شود.

از نظر کارایی، JSON-RPC نسبت به فناوری های باینری مانند gRPC ضعیف تر است، زیرا پیام های JSON حجیم تر هستند و پردازش آنها به دلیل متنی بودن، سربار بیشتری دارد. با این حال، در بسیاری از کاربردهای عملی این کاهش کارایی آن قدر شدید نیست که مانع استفاده شود، به ویژه اگر تعداد درخواست ها کم یا متوسط باشد. بنابراین، JSON-RPC تعادلی میان سادگی و کارایی ایجاد می کند و برای سناریوهایی که سرعت توسعه و سهولت کار با پیام ها مهم تر از بیشینه سازی راندمان است، انتخاب مناسبی به شمار می آید.

در حوزه ی موارد استفاده، JSON-RPC برای Simple APIs، ابزارهای سبک، افزونه ها، سرویس های داخلی ساده و پروژه هایی که نیاز به خوانایی بالا و تنظیمات کم دارند، گزینه ای بسیار مناسب است. نقطه ی قوت اصلی آن در سادگی، انعطاف و سهولت پیاده سازی است، اما در مقایسه با gRPC از

نظر قرارداد رسمی، Streaming و کارایی در مقیاس بالا محدودیت‌هایی دارد.

۳.۲.۹ XML-RPC

XML-RPC یکی از قدیمی‌ترین فناوری‌های شناخته‌شده در حوزه‌ی RPC است که از XML برای نمایش و انتقال داده‌ها استفاده می‌کند. این فناوری در زمانی مطرح شد که نیاز به تبادل ساخت‌یافته‌ی داده میان سیستم‌های مختلف وجود داشت و XML به عنوان یک قالب استاندارد، قابل حمل و مستقل از سکو، نقش مهمی در یکپارچه‌سازی سامانه‌ها ایفا می‌کرد. به همین دلیل، XML-RPC در بسیاری از سامانه‌های قدیمی و زیرساخت‌های Legacy هنوز هم دیده می‌شود. ویژگی اصلی XML-RPC در سادگی مفهومی آن است؛ یعنی همان ایده‌ی فراخوانی متد از راه دور با استفاده از پیام‌های ساخت‌یافته‌ی XML. با این حال، در مقایسه با فناوری‌های جدیدتر، این رویکرد محدودیت‌های قابل توجهی دارد. نخست آن که XML نسبت به قالب‌های مدرن‌تر، بسیار پرحجم‌تر است و به همین دلیل سربار ارتباطی بالاتری ایجاد می‌کند. دوم آن که پردازش و Parsing پیام‌های XML معمولاً پرهزینه‌تر و کندتر از پردازش قالب‌هایی مانند JSON یا Protobuf است. این دو عامل باعث می‌شوند XML-RPC برای محیط‌های پرتراфик و سامانه‌های با نیاز کارایی بالا مناسب نباشد.

از نظر قرارداد رابط، XML-RPC فاقد ساختار رسمی و مدرن مشابه IDL‌های امروزی است و به همین دلیل توسعه و نگهداری آن در سامانه‌های بزرگ ممکن است دشوارتر باشد. همچنین این فناوری معمولاً از قابلیت‌های پیشرفته‌ای مانند Streaming پشتیبانی نمی‌کند و انعطاف‌پذیری آن در مقایسه با فناوری‌های مدرن کمتر است. با این وجود، XML-RPC هنوز در برخی محیط‌ها ارزش خود را حفظ کرده است، به‌ویژه زمانی که یک سامانه باید با زیرساخت‌های قدیمی، محصولات سازمانی قدیمی‌تر، یا سرویس‌هایی که از قبل بر پایه‌ی XML ساخته شده‌اند یکپارچه شود.

در نتیجه، XML-RPC را می‌توان فناوری‌ای قابل اعتماد اما قدیمی دانست که بیشتر برای سازگاری با سامانه‌های موجود و محیط‌های Legacy مناسب است. اگرچه در پروژه‌های جدید معمولاً گزینه‌ی اول نیست، اما در سناریوهایی که سازگاری تاریخی و تداوم کارکرد با سیستم‌های قدیمی اهمیت

دارد، همچنان می تواند قابل استفاده باشد.

۳.۹ جدول مقایسه توسعه یافته

Streaming	IDL	Data Format	Technology
Yes	Yes	Protobuf	gRPC
No	No/Optional	JSON	JSON-RPC
No	No	XML	XML-RPC

Efficiency	Use Case	Language Support	Technology
Very High	Microservices	Very High	gRPC
Medium	Simple APIs	High	JSON-RPC
Low	Legacy Systems	High	XML-RPC

۴.۹ تحلیل نهایی

با جمع بندی معیارهای بررسی شده می توان گفت که هر یک از فناوری های مورد مطالعه برای نوع خاصی از نیازها و سناریوهای عملیاتی مناسب هستند و هیچ گزینه ای به صورت مطلق در همه ی شرایط بهترین انتخاب نیست. انتخاب فناوری مناسب باید بر اساس عواملی مانند مقیاس سامانه، نوع بار کاری، نیاز به کارایی بالا، پیچیدگی توسعه، نیاز به قرارداد رسمی و سطح سازگاری با زیرساخت های موجود انجام شود.

از منظر کارایی، gRPC در رتبه ی بالاتری قرار می گیرد، زیرا استفاده از فرمت باینری Protobuf و پروتکل HTTP/2 باعث می شود حجم پیام ها کمتر، سرعت تبادل بیشتر و مصرف منابع بهینه تر باشد. این مزیت در سیستم های توزیع شده ای که میان سرویس ها حجم بالایی از فراخوانی ها رد و بدل می شود، اهمیت زیادی پیدا می کند. در مقابل، XML-RPC به دلیل استفاده از XML بیشترین سربار را دارد و در محیط هایی که کارایی بسیار مهم است، گزینه ی مطلوبی محسوب نمی شود.

JSON-RPC در این میان جایگاه میانی دارد؛ یعنی از XML-RPC سبک تر و ساده تر است، اما همچنان به دلیل متنی بودن پیام ها، به کارایی gRPC نمی رسد. از نظر سادگی توسعه، JSON-RPC معمولاً راحت ترین گزینه است. این فناوری نیازمند زنجیره ای ابزار پیچیده یا فرایند تولید کد نیست و توسعه دهنده می تواند تنها با استفاده از ساختارهای متداول JSON یک سرویس قابل استفاده ایجاد کند. این مزیت برای پروژه های کوچک، نمونه های اولیه و سرویس هایی که باید در زمان کوتاه راه اندازی شوند، بسیار ارزشمند است. در مقابل، gRPC به دلیل وابستگی به IDL، ابزارهای تولید کد و ساختار رسمی تر، راه اندازی دشوارتری دارد، اما همین پیچیدگی اولیه در پروژه های بزرگ تر به مزیت تبدیل می شود، زیرا باعث انسجام بیشتر و کاهش خطاهای نگهداری می گردد.

از منظر توسعه پذیری و نگهداری بلندمدت، gRPC انتخاب مناسب تری است. قرارداد رسمی مبتنی بر proto باعث می شود تغییرات سرویس ها بهتر کنترل شوند، وابستگی میان کلاینت و سرور شفاف تر باشد و فرایند نسخه پذیری با ریسک کمتری انجام شود. این ویژگی در تیم های بزرگ و سامانه هایی که در طول زمان به روزرسانی های زیادی دارند، بسیار مهم است. در مقابل، در فناوری هایی که قرارداد رسمی ضعیف تر یا اختیاری است، مانند JSON-RPC، انعطاف بالا با هزینه ای افزایش احتمال ناسازگاری همراه می شود.

از نظر تناسب با سیستم های توزیع شده، gRPC به دلیل پشتیبانی از Streaming، سرعت بالا، ساختارمندی و امکان استفاده در چند زبان مختلف، گزینه ای بسیار مناسب برای ارتباط داخلی میان سرویس ها و ماشین های مجازی است. در مقابل، JSON-RPC برای ارتباطات ساده تر، ابزارهای سبک و API هایی که پیچیدگی زیادی ندارند انتخاب مطلوب تری است. XML-RPC نیز هرچند در معماری های مدرن کمتر توصیه می شود، اما همچنان در سناریوهایی که نیاز به سازگاری با سامانه های قدیمی وجود دارد، ارزش عملیاتی خود را حفظ می کند.

در نهایت، اگر هدف طراحی یک سامانه ای توزیع شده ای مدرن، مقیاس پذیر و کارآمد باشد، gRPC معمولاً بهترین انتخاب برای ارتباطات داخلی خواهد بود. اگر سادگی، خوانایی پیام ها و راه اندازی سریع اولویت بالاتری داشته باشد، JSON-RPC انتخاب معقول تری است. همچنین اگر محدودیت

اصلی، سازگاری با زیرساخت ها و سرویس های قدیمی باشد، XML-RPC همچنان می تواند به عنوان یک گزینه ی قابل قبول مطرح شود.

مراجع